

Actividad_Extra04b

January 28, 2026

1 Práctica: Uso de Formatters en Python

1.1 Nombre : Luis Lema

1.2 Fecha: 28/01/2026

1.3 Link Repo: https://github.com/LuisALema/Metodos_Numericos_2025B/tree/main/T

1.4 Introducción

Los formatters en Python permiten mantener un código legible, ordenado y acorde a estándares como PEP8. Estas herramientas automatizan el formateo del código sin alterar su funcionamiento, facilitando el trabajo colaborativo y el mantenimiento del software.

1.5 Objetivo

Analizar y comparar diferentes formatters populares en Python aplicados sobre un mismo archivo de código, con el fin de identificar sus características y diferencias principales.

1.6 Herramientas Utilizadas

- Python 3
- Jupyter Notebook
- Black
- autopep8
- YAPF
- isort

1.7 Código base de prueba

El siguiente código presenta problemas de formato y estilo, los cuales serán corregidos mediante distintos formatters.

```
[1]: %%writefile formatter_test.py
import math,sys
```

```
def calcular(a,b ):
    x= a+b
    y=a*b
    if(x>10):print("Mayor a 10")
    else:
        print("Menor o igual a 10")
    return(x,y)

print(    calcular(5,8))
```

Writing formatter_test.py

1.8 Instalación de Formatters

Para realizar la práctica, se instalaron los siguientes formatters mediante pip.

```
[2]: !pip install black autopep8 yapf isort
```

```
Collecting black
  Downloading black-26.1.0-cp312-cp312-win_amd64.whl.metadata (88 kB)
Collecting autopep8
  Downloading autopep8-2.3.2-py2.py3-none-any.whl.metadata (16 kB)
Collecting yapf
  Downloading yapf-0.43.0-py3-none-any.whl.metadata (46 kB)
Collecting isort
  Downloading isort-7.0.0-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: click>=8.0.0 in
c:\users\bzbzb\appdata\local\programs\python\python312\lib\site-packages (from
black) (8.3.1)
Collecting mypy_extensions>=0.4.3 (from black)
  Downloading mypy_extensions-1.1.0-py3-none-any.whl.metadata (1.1 kB)
Requirement already satisfied: packaging>=22.0 in
c:\users\bzbzb\appdata\local\programs\python\python312\lib\site-packages (from
black) (25.0)
Collecting pathspec>=1.0.0 (from black)
  Downloading pathspec-1.0.4-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: platformdirs>=2 in
c:\users\bzbzb\appdata\local\programs\python\python312\lib\site-packages (from
black) (4.5.0)
Collecting pytokens>=0.3.0 (from black)
  Downloading pytokens-0.4.0-cp312-cp312-win_amd64.whl.metadata (3.9 kB)
Collecting pycodestyle>=2.12.0 (from autopep8)
  Downloading pycodestyle-2.14.0-py2.py3-none-any.whl.metadata (4.5 kB)
Requirement already satisfied: colorama in
c:\users\bzbzb\appdata\local\programs\python\python312\lib\site-packages (from
click>=8.0.0->black) (0.4.6)
Downloading black-26.1.0-cp312-cp312-win_amd64.whl (1.4 MB)
----- 0.0/1.4 MB ? eta -:-:--
----- 0.8/1.4 MB 4.2 MB/s eta 0:00:01
```

```

----- 1.4/1.4 MB 4.6 MB/s 0:00:00
Downloading autopep8-2.3.2-py2.py3-none-any.whl (45 kB)
Downloading yapf-0.43.0-py3-none-any.whl (256 kB)
Downloading isort-7.0.0-py3-none-any.whl (94 kB)
Downloading mypy_extensions-1.1.0-py3-none-any.whl (5.0 kB)
Downloading pathspec-1.0.4-py3-none-any.whl (55 kB)
Downloading pycodestyle-2.14.0-py2.py3-none-any.whl (31 kB)
Downloading pytokens-0.4.0-cp312-cp312-win_amd64.whl (102 kB)
Installing collected packages: yapf, pytokens, pycodestyle, pathspec, mypy-
extensions, isort, black, autopep8

```

```

----- 0/8 [yapf]
----- 2/8 [pycodestyle]
----- 5/8 [isort]
----- 6/8 [black]
----- 6/8 [black]
----- 8/8 [autopep8]

```

Successfully installed autopep8-2.3.2 black-26.1.0 isort-7.0.0 mypy-extensions-1.1.0 pathspec-1.0.4 pycodestyle-2.14.0 pytokens-0.4.0 yapf-0.43.0

1.9 Formatter Black

Black aplica un formato estricto y automático, imponiendo un estilo uniforme sin necesidad de configuración manual.

```

[9]: !black formatter_test.py

with open("formatter_test.py", "r", encoding="utf-8") as f:
    print(f.read())

```

```

import math
import sys

```

```

def calcular(a, b):
    x = a+b
    y = a*b
    if (x > 10):
        print("Mayor a 10")
    else:
        print("Menor o igual a 10")
    return (x, y)

```

```

print(calcular(5, 8))

```

```

error: cannot format formatter_test.py: Cannot parse: 7:0:      y = a*b

```

```
Oh no!  
1 file failed to reformat.
```

1.10 Formatter autopep8

autopep8 corrige errores de estilo de acuerdo con la norma PEP8, manteniendo la estructura original del código.

```
[10]: !autopep8 --in-place formatter_test.py  
  
with open("formatter_test.py", "r", encoding="utf-8") as f:  
    print(f.read())
```

```
import math  
import sys
```

```
def calcular(a, b):  
    x = a+b  
    y = a*b  
    if (x > 10):  
        print("Mayor a 10")  
    else:  
        print("Menor o igual a 10")  
    return (x, y)
```

```
print(calcular(5, 8))
```

1.11 Formatter YAPF

YAPF permite estilos configurables y genera un código con mayor flexibilidad visual.

```
[11]: !yapf -i formatter_test.py  
  
with open("formatter_test.py", "r", encoding="utf-8") as f:  
    print(f.read())
```

```
import math  
import sys
```

```
def calcular(a, b):  
    x = a+b  
    y = a*b  
    if (x > 10):  
        print("Mayor a 10")
```

```
    else:
        print("Menor o igual a 10")
    return (x, y)
```

```
print(calcular(5, 8))
```

```
yapf: formatter_test.py:7:5: unexpected indent
```

1.12 Formatter isort

isort ordena y organiza automáticamente las sentencias de importación del archivo Python.

```
[12]: !isort formatter_test.py

with open("formatter_test.py", "r", encoding="utf-8") as f:
    print(f.read())
```

```
import math
import sys
```

```
def calcular(a, b):
    x = a+b
    y = a*b
    if (x > 10):
        print("Mayor a 10")
    else:
        print("Menor o igual a 10")
    return (x, y)
```

```
print(calcular(5, 8))
```